

Reviewer Pack — Additive Energy Lower Bound via Read-Twice Branching Program...

Entry 1de2d7187fe1 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 07:07:22 UTC

Additive Energy Lower Bound via Read-Twice Branching Program Discrepancy

Entry ID: 1de2d7187fe1 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-10 07:07:22 UTC

1. Conjecture

Field A (mathematical branch): Additive Combinatorics **Field B** (complexity object): Read-Twice Branching Programs

Statement:

For every read-twice branching program P computing a function $f: \{0,1\}^n \rightarrow \{-1,1\}$, the additive energy $E(f)$ of f is at least $\Omega(n)$ times the discrepancy $\text{disc}(P)$ of P .

Rationale (proposer's reasoning):

Additive energy measures the number of additive configurations in a function, which could expose structural constraints on functions computable by read-twice BPs. If read-twice BPs have limited additive structure, their discrepancy (a measure of imbalance) might inversely correlate with additive energy, suggesting a nontrivial relationship between algebraic structure and computational complexity.

Taxonomy category: DISPERSION_DISCREPANCY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): bfaff13147d10cc5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def generate_random_read_twice_bp(n):
    bp = []
    for i in range(2**n):
        if i % 2 == 0:
            bp.append(random.choice([-1, 1]))
        else:
            bp.append(-bp[i-1])
    return bp

def compute_function(bp, n):
    f = [0] * (2**n)
    for i in range(2**n):
        f[i] = bp[i]
    return f

def compute_additive_energy(f, n):
    energy = 0
    for x in range(2**n):
        for y in range(x+1, 2**n):
            for z in range(y+1, 2**n):
                for w in range(z+1, 2**n):
                    if f[x] + f[y] == f[z] + f[w]:
                        energy += 1
    return energy

def compute_discrepancy(bp, n):
    max_disc = 0
    for S in range(1, 2**n):
        sum_S = sum(bp[i] for i in range(n) if (S >> i) & 1)
        disc = abs(sum_S - (2**n - S))
        if disc > max_disc:
            max_disc = disc
    return max_disc

def run_trial(seed: int) -> dict:
    random.seed(seed)
```

```

n = random.choice([5, 10, 15, 20, 30, 40])
bp = generate_random_read_twice_bp(n)
f = compute_function(bp, n)
energy = compute_additive_energy(f, n)
disc = compute_discrepancy(bp, n)

C = 1.0 # Empirical constant
metric_value = energy / (C * n * disc) if disc != 0 else float('inf')

return {
    "metric_name": "additive_energy_over_disc",
    "metric_value": metric_value,
    "instances_tested": 1,
    "conjecture_holds": metric_value >= 1.0, # Adjust C as needed
    "counterexample": "" if metric_value >= 1.0 else "discrepancy_too_small"
}

if __name__ == "__main__":
    seeds = list(map(int, sys.argv[1:])) or [2**i + 1 for i in range(5, 8)] # Default to first 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"discrepancy_too_small\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results, preventing evaluation of support fraction or counterexamples. | next: Run the test with extended time limits and verify hardware/software constraints

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	55475
2	propose	ollama_remote	qwen3:8b	0	0	111495
3	preregistration	ollama_remote	qwen3:8b	0	0	24461
4	novelty	ollama_remote	qwen3:8b	0	0	20689
5	novelty	ollama_remote	qwen3:8b	0	0	17184
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14183
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8920
8	judge	ollama_remote	qwen3:8b	0	0	15891

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 268297 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/1de2d7187fe1.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1de2d7187fe1.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/1de2d7187fe1.tar.gz` (if generated)